



Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2025/2026

SolariUM

Criação de um motor gráfico

Luís Ferreira
a98286

José Fernandes
a106937

Gabriel Dantas
a107291

Simão Oliveira
a107322

Abril, 2026

Índice

1. Introdução	1
2. Generator	2
2.1. Superfícies de Bézier (<i>patches</i>)	2
2.1.1. Motivação e Contexto	2
2.1.2. Formato dos Ficheiros de <i>Patch</i>	2
2.1.3. Parametrização e Geração de Vértices	2
2.1.4. Interface de Linha de Comandos	4
2.1.5. Implementação	4
3. Engine	6
3.1. <i>Vertex Buffer Objects</i> (VBOs)	6
3.1.1. Motivação	6
3.1.2. Implementação	6
3.1.2.1. Abordagem com Vertex Array Objects (VAOs)	6
3.1.2.2. Estrutura de Dados	7
3.1.2.3. Carregamento	8
3.1.2.4. Renderização	9
3.2. Curvas de Catmull-Rom	10
3.2.1. Motivação	10
3.2.2. Estrutura no XML	11
3.2.3. Implementação	12
3.2.3.1. Avaliação da Curva	12
3.2.3.2. Alinhamento com a Tangente	12
3.2.4. Rotações Dependentes do Tempo	13
3.2.5. Visualização e Modo de Depuração	13
3.3. <i>Engine</i> e Gestão do Tempo	14
3.4. Estrutura de Transformações Atualizada	15
3.5. Gestão de Memória e Ciclo de Vida dos Modelos	15
3.6. Sistema de Logging de Desempenho (Framelog)	16
3.6.1. Interface de Linha de Comandos	16
4. Modelo do Sistema Solar Animado	18
4.1. Órbitas e Períodos	18
4.2. Rotações Próprias	18
4.3. Estrutura do XML Atualizada	19
4.3.1. Cometas com Trajetória Curva	20
4.3.2. Cinturões de asteróides	20
4.4. Geração Procedimental de Órbitas	21

4.5. Resultados Obtidos	22
5. Conclusão	25
Bibliografia	26
Lista de Siglas e Acrónimos	27
Anexos	28
Anexo 1 Logo da Universidade do Minho	28

Lista de Figuras

Figura 1	Patch de Bezier ilustrativo.	3
Figura 2	Curva de Catmull-Rom bidimensional ilustrativa.	11
Figura 3	Resultado 1 — curvas de Catmull-Rom com 7 pontos.	22
Figura 4	Resultado 2 — curvas de Catmull-Rom com 7 pontos, vista de cima.	22
Figura 5	Resultado 3 — curvas de Catmull-Rom com 7 pontos, vista de cima.	23
Figura 6	Resultado 4 — curvas de Catmull-Rom com 120 pontos.	23
Figura 7	Resultado 5 — curvas de Catmull-Rom com 120 pontos, vista de cima.	23
Figura 8	Resultado 6 — curvas de Catmull-Rom com 120 pontos, vista lateral.	23

1. Introdução

O presente relatório pretende descrever o processo de desenvolvimento do trabalho laboratorial desenvolvido no âmbito da Unidade Curricular de Computação Gráfica.

Com base no enunciado fornecido o objetivo é criar um sistema completo de geração e visualização de cenas tridimensionais, composto por duas aplicações principais: o *generator*, responsável pela criação de modelos geométricos, e a *engine*, encarregue da leitura de ficheiros XML, carregamento de modelos e renderização das cenas utilizando a API *OpenGL* [1]. O trabalho é desenvolvido de forma incremental, estando dividido em quatro fases distintas, cada uma introduzindo novos conceitos fundamentais da Computação Gráfica.

Na 1ª fase, foi implementada a leitura do ficheiro XML no arranque da aplicação e criadas as estruturas de dados adequadas para armazenar a informação da cena em memória. O *generator* produz modelos geométricos básicos e a *engine* carrega-os e desenha-os.

Durante a 2ª fase, foi implementado o suporte a transformações geométricas hierárquicas (translação, rotação e escala), organizando a cena sob a forma de uma árvore de grupos. Aplicaram-se transformações aos modelos e seus subgrupos. O cenário de teste utilizado foi um modelo estático do sistema solar, que se usará para prosseguir com o projeto.

Nesta 3ª fase, o *generator* deve passar a suportar a geração de superfícies a partir de *patches* de Bezier. Na *engine*, devem ser implementadas transformações dependentes do tempo, nomeadamente translações ao longo de curvas Catmull-Rom e rotações contínuas. É ainda mandatório utilizar VBOs para a renderização. O cenário demonstrativo é um sistema solar animado.

2. Generator

2.1. Superfícies de Bézier (*patches*)

2.1.1. Motivação e Contexto

As superfícies de Bézier são uma generalização bidimensional das curvas de Bézier. Uma *patch* de Bézier bicúbica é definida por uma grelha de 4×4 que especifica pontos de controlo, que “atraem” a superfície sem que esta necessariamente passe por eles. A principal vantagem desta representação é a possibilidade de descrever superfícies complexas e suaves com muito poucos parâmetros, sendo amplamente utilizada em modelação geométrica [2].

2.1.2. Formato dos Ficheiros de *Patch*

O *generator* lê ficheiros de *patch* com extensão `.patch`. O formato define-se como:

- Primeira linha: número de *patches* (N);
- As N linhas seguintes: cada linha lista os 16 índices dos pontos de controlo da *patch*, separados por vírgulas, numa ordem linha a linha (4 linhas de 4 índices);
- Uma linha com o número total de pontos de controlo;
- As linhas seguintes: coordenadas (x, y, z) de cada ponto de controlo, um por linha.

Veja-se o seguinte excerto de um ficheiro `.patch` de exemplo:

```
32
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15
3, 16, 17, 18, 7, 19, 20, 21, 11, 22, 23, 24, 15, 25, 26, 27
...
16
1.4, 0.0, 0.0
1.4, 0.933, 0.0
0.933, 1.4, 0.0
...
```

Listagem 1: Excerto de um ficheiro `.patch`.

2.1.3. Parametrização e Geração de Vértices

Uma *patch* de Bézier bicúbica é avaliada nos parâmetros $(u, v) \in [0, 1]^2$. Dado o conjunto de 16 pontos de controlo $P_{i,j}$ com $i, j \in \{0, 1, 2, 3\}$, um ponto na superfície obtém-se por:

$$Q(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_{i(u)} B_{j(v)} P_{i,j}$$

onde $B_{k(t)}$ são os polinómios de Bernstein de 3º grau:

$$B_0(t) = (1 - t)^3, \quad B_1(t) = 3t(1 - t)^2, \quad B_2(t) = 3t^2(1 - t), \quad B_3(t) = t^3$$

O *generator* recebe como parâmetros o ficheiro .patch e o número de *tessellations* (tesselações – isto é, o preenchimento de uma superfície plana com formas geométricas repetidas, sem sobreposições ou lacunas), seja T . Este controla a resolução da malha resultante: o domínio $[0, 1]^2$ é dividido em $T \times T$ células, cada uma correspondendo a um *quad* (dois triângulos). Para cada célula, os parâmetros (u_i, v_j) e (u_{i+1}, v_{j+1}) são calculados com $u_i = \frac{i}{T}$ e $v_j = \frac{j}{T}$.

Os quatro pontos do *quad* são então avaliados na superfície de Bézier e os dois triângulos correspondentes são gerados com a orientação CCW habitual:

$$\begin{cases} p_1 = Q(u_i, v_j) \\ p_2 = Q(u_{i+1}, v_j) \\ p_3 = Q(u_i, v_{j+1}) \\ p_4 = Q(u_{i+1}, v_{j+1}) \end{cases}$$

Na Figura 1 segue-se uma representação de alguns dos elementos descritos, para melhor compreensão. Os pontos de controlo estão assinalados a vermelho e a conexão entre eles marca a forma que vai “atrair” os pontos $Q(u, v)$ a avaliar. Tem-se dois exemplos destes pontos, $Q(0.7, 0.4)$ e $Q(0.8, 0.8)$. T representa uma tesselação, ou seja, um dos *quads* de renderização da figura resultante.

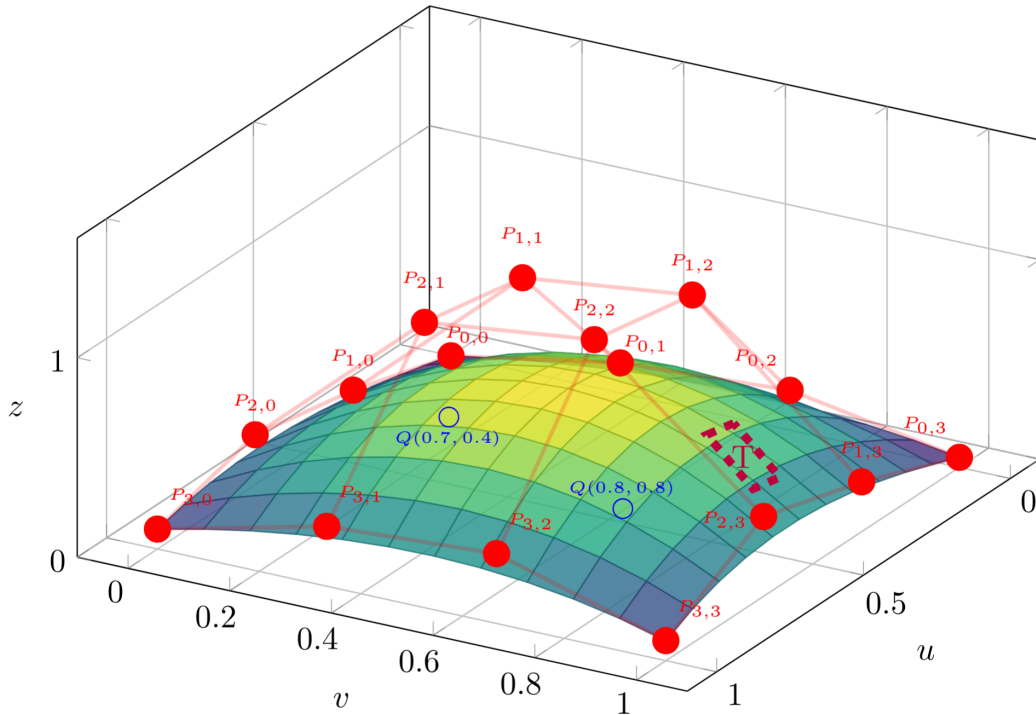


Figura 1: Patch de Bezier ilustrativo.

2.1.4. Interface de Linha de Comandos

O novo comando do *generator* para *patches* de Bézier é:

```
./generator bezier <ficheiro.patch> <tessellations> <output.3d>
```

Por exemplo, para gerar o clássico modelo do bule de Utah com resolução 10:

```
./generator bezier teapot.patch 10 teapot.3d
```

2.1.5. Implementação

A implementação foi dividida em dois passos. Primeiro, o ficheiro *.patch* é lido e os pontos de controlo são carregados para um vector<array<float,3>>. Os índices de cada *patch* são armazenados num vector<array<int,16>>.

De seguida, para cada *patch*, itera-se sobre uma grelha $T \times T$ e avalia-se a superfície para diferentes valores dos parâmetros (u, v) .

A função de avaliação de uma superfície de Bézier bicúbica segue uma abordagem matricial. Para cada par (u, v) , são construídos os vetores de potências:

$$U = [u^3, u^2, u, 1], \quad V = [v^3, v^2, v, 1]$$

Em seguida, estes vetores são multiplicados pela matriz base de Bézier M , obtendo-se:

$$MU = M * U, \quad MV = M * V$$

O ponto final da superfície é então calculado através de uma soma ponderada dos 16 pontos de controlo:

$$P(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 MU[i] * G[i][j] * MV[j]$$

onde G representa a matriz dos pontos de controlo da *patch*. Este processo é aplicado independentemente às coordenadas x , y e z .

```
function evalBezierPatch(cp, u, v):
    U = [u^3, u^2, u, 1]
    V = [v^3, v^2, v, 1]

    MU = M * U
    MV = M * V

    result = (0, 0, 0)
    for i = 0..3:
        for j = 0..3:
            w = MU[i] * MV[j]
            result += w * cp[i][j]
    return result
```

Listagem 2: Pseudocódigo para avaliação de uma *patch* de Bézier bicúbica.

O resultado final é armazenado no formato .3d habitual — cada triângulo é escrito como três linhas de coordenadas, mantendo a compatibilidade com o resto da *pipeline*.

3. Engine

3.1. *Vertex Buffer Objects* (VBOs)

3.1.1. Motivação

Nas fases anteriores, a geometria era enviada à GPU a cada *frame* através do modo imediato do *OpenGL* — `glBegin(GL_TRIANGLES)`, um `glVertex3f` por vértice e `glEnd()`. Esta abordagem, embora simples, é extremamente ineficiente: a cada *frame*, a CPU tem de iterar sobre todos os vértices de todos os modelos, realizar uma chamada ao *driver* por vértice e transferir os dados pela barreira CPU-GPU. Para cenas simples, o impacto é negligenciável, mas para a cena do sistema solar com cinturões de asteroides e milhares de estrelas, o tempo de CPU gasto nesta transferência tornava-se um *bottleneck*.

Os VBOs resolvem este problema ao transferir os dados de geometria para a memória da GPU uma única vez, aquando do carregamento do modelo. Nas *frames* subsequentes, a GPU usa os dados já residentes na sua memória, sem qualquer transferência pela barreira CPU-GPU. O ganho de desempenho é substancial, especialmente para modelos estáticos com muitos vértices.

3.1.2. Implementação

A implementação dos VBOs implicou alterações na estrutura de dados `Model` e na lógica de carregamento e renderização de modelos.

3.1.2.1. Abordagem com *Vertex Array Objects* (VAOs)

Embora nas aulas tenha sido utilizada a abordagem baseada em `glEnableClientState`, correspondente ao *fixed pipeline* do *OpenGL*, nesta implementação optou-se por utilizar *Vertex Array Objects* (VAOs) em conjunto com *Vertex Buffer Objects* (VBOs), seguindo o paradigma moderno do *OpenGL* (*core profile*).

A principal diferença entre as duas abordagens reside na forma como a configuração dos atributos de vértices é gerida. Com `glEnableClientState`, é necessário ativar e especificar explicitamente os arrays de vértices (e outros atributos) a cada chamada de renderização, o que implica um maior número de interações com o *driver* e uma gestão de estado global mais propensa a erros.

Por outro lado, os VAOs permitem encapsular toda essa configuração — incluindo a associação de *buffers* e a definição dos atributos — num único objeto. Desta forma, após uma fase inicial de configuração, a renderização de um modelo reduz-se essencialmente

à ligação do VAO e a uma chamada a *glDrawArrays*, simplificando significativamente o código e reduzindo o número de chamadas ao *driver*.

Esta abordagem não só melhora a organização e legibilidade da implementação, como também proporciona melhores oportunidades de otimização por parte da GPU, sendo atualmente a prática recomendada no desenvolvimento de aplicações gráficas modernas.

3.1.2.2. Estrutura de Dados

A estrutura *Model* foi estendida com o identificador dos *buffers* OpenGL de posição e algumas variáveis auxiliares para facilitar a implementação da utilização dos VBOs:

```
struct Model {
    string file;
    list<Vertex> vertices;
    float r, g, b;
    bool cull;
    // VBO state
    RenderMode renderMode;
    GLuint vaoId;
    GLuint vboId;
    int vertexCount;
    bool isDirty;
    bool gpuReady;

    Model() : r(1.f), g(1.f), b(1.f), cull(true),
             renderMode(STATIC),
             vaoId(0), vboId(0), vertexCount(0),
             isDirty(false), gpuReady(false) {}
};
```

Listagem 3: Estrutura *Model* com suporte a VBOs.

Os campos adicionados para suporte a VBOs têm as seguintes funções:

- *vaoId* — Identificador do *Vertex Array Object* (VAO). Este objeto guarda a configuração dos atributos dos vértices, incluindo a forma como os dados estão organizados no VBO e como são interpretados pelo *shader*.
- *vboId* — Identificador do *Vertex Buffer Object* (VBO), responsável por armazenar na GPU os dados dos vértices (neste caso, posições 3D).
- *vertexCount* — Número total de vértices do modelo. Este valor é utilizado durante a renderização para indicar quantos vértices devem ser processados.
- *renderMode* — Define o tipo de utilização do buffer (STATIC ou DYNAMIC). Este parâmetro influencia a forma como os dados são armazenados na GPU, permitindo otimizações por parte do *driver*.
- *gpuReady* — Indica se os buffers (VAO e VBO) já foram criados na GPU. Evita recriações desnecessárias a cada frame.

- `isDirty` — Indica se os dados do modelo foram alterados na memória principal e precisam de ser novamente enviados para a GPU.

Esta separação entre dados em CPU (vertices) e dados em GPU (`vboId`, `vaoId`) permite uma renderização mais eficiente, uma vez que os dados não necessitam de ser enviados repetidamente a cada frame.

3.1.2.3. Carregamento

Após o carregamento de um modelo para a memória principal, os vértices são convertidos para um formato contíguo e transferidos para a GPU através de *Vertex Buffer Objects* (VBOs), sendo associados a um *Vertex Array Object* (VAO).

O processo começa por transformar a lista de vértices numa sequência linear de valores em vírgula flutuante, adequada para envio eficiente para a GPU.

De seguida, são criados (caso ainda não existam) um VAO e um VBO, que irão armazenar e descrever os dados do modelo no lado da GPU. O tipo de utilização do *buffer* é definido consoante o modo de renderização do modelo.

```
function uploadModelToGPU(model):
    buffer = []

    // Converter vértices para array contíguo
    for each vertex in model.vertices:
        buffer.append(vertex.x)
        buffer.append(vertex.y)
        buffer.append(vertex.z)

    model.vertexCount = number of vertices

    if model.renderMode == STATIC:
        usage = GL_STATIC_DRAW
    else:
        usage = GL_DYNAMIC_DRAW

    if model not yet on GPU:
        create VAO
        create VBO

    bind VAO
    bind VBO

    upload buffer to GPU (glBufferData)

    define vertex attribute (posição 3D)
    enable vertex attribute

    unbind VAO

    mark model as clean
```

Listagem 4: Pseudocódigo do envio de um modelo para a GPU.

O *Vertex Buffer Object* (VBO) é responsável por armazenar os dados dos vértices na memória da GPU, permitindo um acesso eficiente durante o processo de renderização. Neste caso, apenas as posições dos vértices são enviadas.

O *Vertex Array Object* (VAO) guarda a configuração dos atributos dos vértices, incluindo a forma como os dados estão organizados no VBO e como devem ser interpretados pelo *pipeline* gráfico.

A função `glBufferData` realiza a transferência dos dados da memória principal para a GPU. O parâmetro de utilização (`GL_STATIC_DRAW` ou `GL_DYNAMIC_DRAW`) informa o *driver* sobre a frequência de atualização dos dados, permitindo otimizações internas.

Por fim, a função `glVertexAttribPointer` define como os dados devem ser lidos (neste caso, conjuntos de 3 valores correspondentes às coordenadas (x, y, z)), e `glEnableVertexAttribArray` ativa esse atributo para utilização no *shader*.

3.1.2.4. Renderização

A fase de renderização percorre recursivamente a hierarquia de grupos, aplicando transformações e desenhando os modelos associados a cada grupo.

Cada grupo começa por guardar o estado atual da matriz de transformação com `glPushMatrix`, garantindo que as transformações aplicadas não afetam outros ramos da hierarquia.

As transformações são aplicadas sequencialmente:

- Translação (TRANSLATE) — pode ser estática ou animada ao longo de uma curva de Catmull-Rom. No caso animado, a posição é calculada em função do tempo, permitindo movimento contínuo. Opcionalmente, o modelo pode ser alinhado com a tangente da curva.
- Rotação (ROTATE) — pode ser estática ou dependente do tempo, sendo o ângulo atualizado continuamente para criar animações.
- Escala (SCALE) — aplicada diretamente aos eixos do modelo.

Após aplicar as transformações, cada modelo do grupo é desenhado.

```
function renderGroup(group):  
    push matrix  
  
    apply all transformations (translate / rotate / scale)  
  
    for each model in group:  
        if culling disabled:  
            disable face culling  
  
        set model color
```

```

if VBO enabled:

    if model not in GPU or needs update:
        uploadModelToGPU(model)

    bind VAO
    draw (glDrawArrays)
    unbind VAO

else:
    begin GL_TRIANGLES
    for each vertex:
        send vertex
    end

    restore culling state if needed

for each child group:
    renderGroup(child)

pop matrix

```

Listagem 5: Pseudocódigo do processo de renderização com suporte a VBOs e hierarquia.

Quando os VBOs estão ativos, a renderização é realizada através da ligação do *Vertex Array Object* (VAO) e de uma única chamada a `glDrawArrays`, que processa todos os vértices diretamente na GPU.

Caso contrário, é utilizado um modo de compatibilidade (*fallback*) onde os vértices são enviados individualmente com `glBegin/glEnd`, o que é significativamente menos eficiente.

A utilização de VAOs permite encapsular toda a configuração dos atributos de vértices, reduzindo o número de chamadas ao *driver* e simplificando o código de renderização.

Além disso, a verificação do estado (`gpuReady` e `isDirty`) evita transferências desnecessárias de dados para a GPU, melhorando o desempenho global da aplicação.

3.2. Curvas de Catmull-Rom

3.2.1. Motivação

As curvas de Catmull-Rom são *splines* que interpolam um conjunto de pontos de controlo, ou seja, a curva passa exatamente por todos os pontos definidos. Esta propriedade de interpolação distingue-as das curvas de Bézier (que apenas aproximam os pontos de controlo) e torna-as particularmente adequadas para definir trajetórias de animação: o utilizador especifica os pontos por onde o objeto deve passar e a curva conecta-os de forma suave e contínua [3].

A fórmula de um segmento de Catmull-Rom entre os pontos P_1 e P_2 , com os pontos vizinhos P_0 e P_3 , para $t \in [0, 1]$, é:

$$Q(t) = \frac{1}{2} \begin{pmatrix} 1 & t & t^2 & t^3 \end{pmatrix} \begin{pmatrix} 0 & 2 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 2 & -5 & 4 & -1 \\ -1 & 3 & -3 & 1 \end{pmatrix} \begin{pmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{pmatrix}$$

Na Figura 2 encontra-se um exemplo ilustrativo de uma curva deste gênero num espaço bidimensional e a forma como os pontos se comportam.

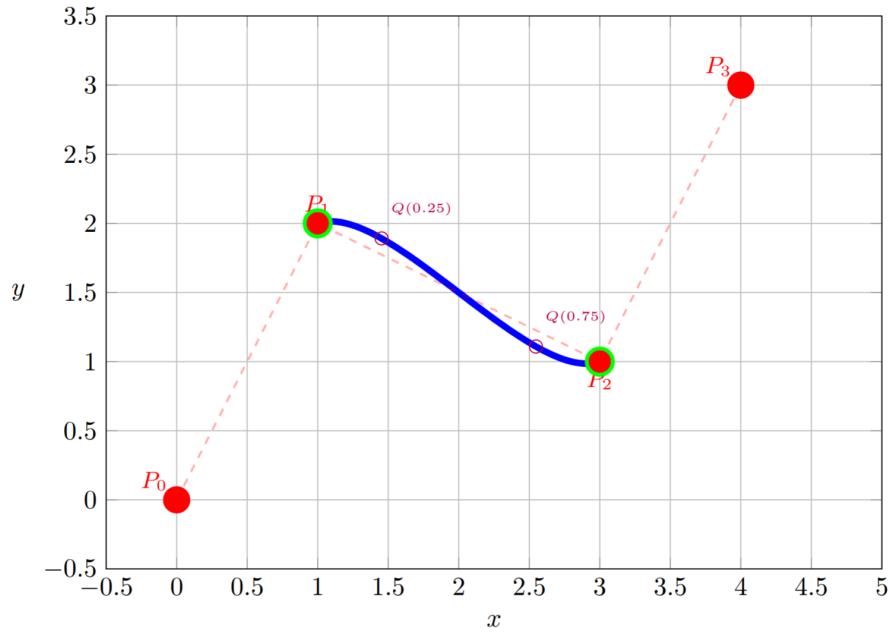


Figura 2: Curva de Catmull-Rom bidimensional ilustrativa.

3.2.2. Estrutura no XML

As translações dependentes do tempo são declaradas no XML através de um elemento `<translate>` com o atributo `time` e uma lista de pontos filhos:

```
<translate time="5" align="true">
  <point x="1" y="0" z="0"/>
  <point x="0" y="1" z="0"/>
  <point x="-1" y="0" z="0"/>
  <point x="0" y="-1" z="0"/>
  <point x="1" y="0" z="0"/>
  <point x="0" y="1" z="0"/>
  <point x="-1" y="0" z="0"/>
</translate>
```

Listagem 6: Declaração de uma translação ao longo de uma curva Catmull-Rom no XML.

O atributo `time` define o período orbital em segundos — o tempo que o objeto demora a percorrer a curva completa. O atributo `align` indica se o objeto deve ser orientado segundo a tangente da curva em cada instante.

Na implementação utilizada, o número de segmentos da curva de Catmull-Rom depende diretamente do número de pontos de controlo, sendo dado por $N - 3$, onde N representa o número total de pontos.

Cada segmento da curva é definido por quatro pontos consecutivos, sendo avaliado no intervalo entre o segundo e o terceiro ponto. Assim, ao aumentar o número de pontos, aumenta-se o número de segmentos que compõem a trajetória.

No contexto das órbitas do sistema solar, cada segmento representa apenas uma parte da trajetória completa. Como a implementação não considera automaticamente a curva como cíclica (isto é, não existe ligação implícita entre o último e o primeiro ponto), é necessário definir pontos suficientes para cobrir toda a órbita.

Desta forma, a utilização de 7 pontos permite obter 4 segmentos ($7 - 3 = 4$), o que é suficiente para aproximar uma trajetória fechada e visualmente contínua. Com menos pontos, a curva resultante representa apenas uma fração da órbita total.

Assim, a necessidade de múltiplos pontos não decorre da definição matemática da curva de Catmull-Rom, mas sim da forma como a mesma foi implementada, em particular da ausência de fecho automático da curva.

3.2.3. Implementação

3.2.3.1. Avaliação da Curva

Para um dado instante de tempo absoluto t_{abs} (em segundos), o parâmetro global na curva (tempo normalizado) é:

$$t_g = \frac{t_{\text{abs}} \% T}{T}$$

onde T é o período orbital. Este valor é mapeado para o segmento e parâmetro local correspondentes:

$$i = \lfloor t_g \cdot N \rfloor, \quad t_l = t_g \cdot N - i$$

Os quatro pontos de controlo relevantes são então $P_{i-1}, P_i, P_{i+1}, P_{i+2}$ (com índices módulo N) e a posição é avaliada pela fórmula de Catmull-Rom apresentada acima.

3.2.3.2. Alinhamento com a Tangente

Quando o atributo `align` está ativo, o objeto é também orientado segundo a direção de movimento em cada instante. A tangente da curva é calculada analiticamente:

$$\hat{X} = \frac{Q'(t)}{\|Q'(t)\|}$$

onde $Q'(t)$ é a derivada da curva de Catmull-Rom em ordem a t :

$$Q'(t) = \frac{1}{2} \begin{pmatrix} 0 & 1 & 2t & 3t^2 \end{pmatrix} \begin{pmatrix} 0 & 2 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 2 & -5 & 4 & -1 \\ -1 & 3 & -3 & 1 \end{pmatrix} \begin{pmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{pmatrix}$$

Com a tangente $\hat{X}$ calculada, constrói-se a matriz de rotação do objeto, sendo necessário definir dois eixos adicionais. O eixo $\hat{Z}$ é calculado como o produto vetorial do up global (inicialmente $(0, 1, 0)$) com $\hat{X}$, e o eixo $\hat{Y}$ como o produto vetorial de $\hat{X}$ com $\hat{Z}$:

$$\hat{Z} = \frac{\hat{u} \times \hat{X}}{\|\hat{u} \times \hat{X}\|}, \quad \hat{Y} = \hat{X} \times \hat{Z}$$

Esta base ortonormal $(\hat{X}, \hat{Y}, \hat{Z})$ define uma matriz de rotação 4.4 que é aplicada ao objeto através de `glMultMatrixf`, posicionando-o tangencialmente à curva em cada instante. O vetor up é atualizado a cada *frame* usando o $\hat{Y}$ calculado anteriormente.

3.2.4. Rotações Dependentes do Tempo

Para além das translações, as rotações passam também a poder ser dependentes do tempo. Uma rotação animada é declarada no XML com o atributo `time`:

```
<rotate time="10" x="0" y="1" z="0"/>
```

Neste exemplo, o objeto completa uma rotação completa em 10 segundos em torno do eixo Y. O ângulo em cada instante é calculado como:

$$\alpha(t) = \frac{t_{\text{abs}} \% T}{T} \cdot 360^\circ$$

onde T é o período de rotação em segundos. A implementação usa `glutGet(GLUT_ELAPSED_TIME)` para obter o tempo decorrido em milissegundos desde o início da aplicação, convertendo-o para segundos com precisão de vírgula flutuante.

3.2.5. Visualização e Modo de Depuração

Para facilitar o desenvolvimento e validação das trajetórias definidas por curvas de Catmull-Rom, foi implementado um modo de depuração que permite visualizar diretamente a curva na cena.

Neste modo, a curva é amostrada em múltiplos pontos ao longo do intervalo $t \in [0, 1]$ e desenhada como uma polilinha utilizando `GL_LINE_STRIP`. Esta abordagem permite observar a suavidade da interpolação e verificar rapidamente se os pontos de controlo estão corretamente definidos.

```
static void drawCatmullRomCurve(const vector<array<float,3>>& pts) {
    const int samples = 200;

    glDisable(GL_CULL_FACE);
    glColor3f(0.2f, 0.9f, 1.0f);
```

```

glBegin(GL_LINE_STRIP);
for (int i = 0; i <= samples; ++i) {
    float t = (float)i / (float)samples;
    float pos[3];
    catmullRomPoint(pts, t, pos, nullptr);
    glVertex3f(pos[0], pos[1], pos[2]);
}
glEnd();

if (enableCulling) glEnable(GL_CULL_FACE);
}

```

Listagem 7: Função responsável pelo desenho da curva de Catmull-Rom em modo de depuração.

A ativação deste modo é feita através de uma opção de interface (tecla P, descrito no *menu* da aplicação), permitindo ligar e desligar a visualização da curva em tempo real.

Durante a renderização, os valores de amostragem são suficientemente elevados (tipicamente 200 pontos) para garantir uma curva visualmente suave, mesmo em trajetórias com curvaturas mais acentuadas.

Este mecanismo revelou-se especialmente útil na validação das órbitas do sistema solar e das trajetórias dos cometas, permitindo identificar rapidamente erros na disposição dos pontos de controlo ou descontinuidades na curva.

3.3. Engine e Gestão do Tempo

A gestão do tempo na *engine* é baseada no tempo absoluto fornecido pela função `glutGet(GLUT_ELAPSED_TIME)`, que devolve o tempo decorrido desde o início da execução da aplicação.

Este valor é convertido para segundos e utilizado diretamente nas transformações animadas, como translações ao longo de curvas de Catmull-Rom e rotações dependentes do tempo.

```

float elapsed =
    glutGet(GLUT_ELAPSED_TIME) / 1000.0f;

float tNorm =
    fmod(elapsed, t.time) / t.time;

float angle =
    fmod(elapsed, t.time) / t.time * 360.0f;

```

Listagem 8: Utilização do tempo absoluto para cálculo de animações.

A utilização de tempo absoluto permite garantir que a animação é independente da taxa de *frames*. Em vez de depender do tempo decorrido entre *frames*, a posição e

orientação dos objetos são calculadas diretamente em função do tempo global, tornando o comportamento determinístico.

Para animações cíclicas, como órbitas e rotações contínuas, é utilizada a função `fmod`, que permite repetir o movimento ao longo de um período definido (`t.time`). O valor resultante é normalizado para o intervalo $[0, 1]$, sendo depois usado como parâmetro nas funções de interpolação.

O *idle callback* do GLUT é utilizado para garantir atualização contínua da cena:

```
glutIdleFunc(renderScene);
```

Desta forma, a *engine* assegura que as animações são suaves, contínuas e consistentes, independentemente do desempenho do sistema.

3.4. Estrutura de Transformações Atualizada

A estrutura `Transform` foi estendida para suportar transformações dependentes do tempo:

```
struct Transform {
    TransformType type;
    float x, y, z;
    float angle;           // ângulo estático (ROTATE)
    float time;            // período em segundos (0 = estático)
    bool align;            // alinhar com tangente (TRANSLATE)
    vector<array<float,3>> catmullPoints; // pontos Catmull-Rom
};
```

Listagem 9: Estrutura `Transform` estendida com suporte a animações.

A lógica de despacho na função `applyTransform` distingue entre transformações estáticas (campo `time == 0`) e animadas (`time > 0`), aplicando a lógica adequada em cada caso sem alterar a interface com o resto da *engine*.

3.5. Gestão de Memória e Ciclo de Vida dos Modelos

A gestão de memória neste projeto é dividida entre CPU e GPU. Na CPU, os modelos carregados a partir de ficheiros `.3d` são armazenados numa cache (`modelCache`), evitando leituras repetidas do disco. A função `getModelVertices` assegura este comportamento, reutilizando dados caso o modelo já tenha sido carregado.

Na GPU, os dados dos modelos são transferidos para *buffers* através da função `uploadModelToGPU`, que cria um VAO e um VBO caso ainda não existam. Estes buffers são atualizados sempre que o modelo é marcado como “dirty”.

A libertação de memória GPU é feita explicitamente com a função `freeModelGPU`, que remove corretamente o VAO e o VBO associados a cada modelo, evitando fugas de memória.

```
void freeModelGPU(Model& m) {
    if (!m.gpuReady) return;
    glDeleteVertexArrays(1, &m.vaoId);
    glDeleteBuffers(1, &m.vboId);
    m.gpuReady = false;
}
```

Listagem 10: Libertação dos recursos GPU de um modelo.

A função `clearModelCache` apenas limpa a cache da CPU, sendo a libertação da GPU tratada separadamente. Esta separação permite um controlo mais seguro do ciclo de vida dos recursos, especialmente no recarregamento dinâmico da cena.

3.6. Sistema de Logging de Desempenho (Framelog)

Com o objetivo de verificar quantitativamente o ganho de desempenho proporcionado pelos VBOs, foi adicionado um sistema de *logging* de tempos de *frame* integrado na *engine*. Este sistema permite medir o tempo decorrido para cada ciclo de renderização e registar os dados em ficheiro CSV para análise posterior.

3.6.1. Interface de Linha de Comandos

O executável foi estendido com várias opções de linha de comandos relacionadas com a monitorização de desempenho:

<code>--framelog[=output.csv]</code>	Ativa o registo de tempos de frame
<code>--framelog output.csv</code>	Especifica ficheiro de saída
<code>--framelog-count N</code>	Captura exatamente N registos
<code>--no-vbo</code>	Desativa VBOs, força modo imediato
<code>--help, -h, help</code>	Mostra mensagem de ajuda

Exemplos:

```
./engine solar_system.xml
./engine --framelog bench.csv solar_system.xml
./engine --framelog-count 1000 solar_system.xml
./engine --no-vbo --framelog=novbo.csv solar_system.xml
```

Listagem 11: Opções de linha de comandos para logging de desempenho.

A opção `--framelog` ativa a monitorização de tempos de *frame*, registando em ficheiro cada instante de tempo decorrido pela renderização de uma cena completa. Quando não é especificado um caminho explícito, os logs são escritos na pasta `log/` com nomes gerados automaticamente.

A opção `--framelog-count N` limita o número de registos capturados, útil para restringir o tamanho dos ficheiros de benchmark e para comparações rápidas entre modos.

A opção `--no-vbo` é essencial para benchmarking comparativo: desativa completamente o uso de VBOs e força a *engine* a renderizar usando o modo imediato de `glBegin/`

glEnd, reproduzindo o comportamento das fases anteriores. Isto permite medir o desempenho relativo da mesma cena com e sem VBOs.

4. Modelo do Sistema Solar Animado

4.1. Órbitas e Períodos

As órbitas dos planetas foram definidas com quatro pontos de controlo cada, distribuídos num quadrado inscrito na órbita, de modo a que a curva Catmull-Rom resultante se aproxime de um círculo. Os períodos foram escalados de forma que a Terra complete uma volta em 36.5 segundos (um segundo de simulação corresponde a dez dias reais), e os restantes planetas têm períodos proporcionais às suas razões reais [4]:

Planeta	Período real (anos)	Período simulado (s)
Mercúrio	0.24	8.8
Vénus	0.62	22.6
Terra	1.00	36.5
Marte	1.88	68.6
Júpiter	11.86	433
Saturno	29.46	1075
Urano	84.01	3066
Neptuno	164.80	6015
Plutão	248.10	9056
Lua	0.75	27.4

Tabela 1: Períodos orbitais dos planetas no modelo animado.

Para calcular o período orbital simulado, portanto, usou-se a fórmula:

$$T_{\text{rot},s} = 36.5 \times T_{\text{orb},r}$$

Sendo $T_{\text{orb},s}$ o período orbital simulado em segundos e $T_{\text{orb},r}$ o período orbital real em anos.

4.2. Rotações Próprias

Cada planeta possui também uma rotação própria em torno do seu eixo, com um período diferente do orbital. Calcularam-se, novamente, com base em informação real [5], fazendo a razão para a correspondência entre um segundo de simulação para dez dias reais.

O período rotacional simulado, $T_{\text{rot},s}$, foi calculado com base na fórmula:

$$T_{\text{rot},s} = T_{\text{rot},r}/10$$

Sendo $T_{\text{rot},r}$ o período de rotação real.

Assim, temos a tabela:

Planeta	Período real (dias)	Direção	Período simulado (s)
Sol	25.38	Direta	2.54
Mercúrio	58.65	Direta	5.87
Vénus	243.02	Retrógrada	24.30
Terra	1.00	Direta	0.10
Marte	1.03	Direta	0.10
Júpiter	0.41	Direta	0.04
Saturno	0.44	Direta	0.04
Urano	0.72	Retrógrada	0.07
Neptuno	0.67	Direta	0.07
Plutão	6.39	Retrógrada	0.64
Lua	27.32	Direta	2.73

Tabela 2: Períodos rotacionais dos corpos no modelo animado (escala: 10 dias = 1 s).

A informação de direção é representada pelo eixo de rotação. Para os planetas que têm uma direção direta, usa-se o eixo $(0, 1, 0)$. Para os que têm uma direção retrógrada usa-se o eixo $(0, -1, 0)$.

4.3. Estrutura do XML Atualizada

O XML do sistema solar foi revisto para incluir transformações dependentes do tempo. As rotações estáticas dos planetas em torno do Sol foram substituídas por translações ao longo de curvas Catmull-Rom que aproximam as órbitas elípticas. Cada planeta possui uma curva com pontos distribuídos de forma a que a órbita seja aproximadamente circular (vista de cima), com um período proporcional ao período orbital real escalado para fins de demonstração.

```
<!-- MERCURY – orbit period 8.8s; rotation period 5.87s (direct) -->
<group>
  <transform>
    <translate time="8.8" align="false">
      <point x="35" y="0" z="0" />
      <point x="0" y="0" z="35" />
      <point x="-35" y="0" z="0" />
      <point x="0" y="0" z="-35" />
      <point x="35" y="0" z="0" />
      <point x="0" y="0" z="35" />
      <point x="-35" y="0" z="0" />
    </translate>
  </transform>
</group>
```

```

        <rotate time="5.87" x="0" y="1" z="0" />
        <scale x="0.38" y="0.38" z="0.38" />
    </transform>
    <models>
        <model file="icosphere.3d" color="#888888" />
    </models>
</group>

```

Listagem 12: Grupo de Mercúrio com translação animada ao longo de uma curva Catmull-Rom e rotação própria.

4.3.1. Cometas com Trajetória Curva

Como cenário demonstrativo adicional, foram adicionados três cometas que seguem uma trajetória elíptica e alongada. A trajetória foi definida com pontos de controle que formam uma elipse de grande excentricidade, com o Sol num dos focos. O alinhamento com a tangente está ativo para este objeto, fazendo com que o cometa aponte sempre na direção do seu movimento, o que cria um efeito visualmente apelativo especialmente nas curvas mais apertadas da trajetória.

```

<group>
    <transform>
        <translate time="14" align="true">
            <point x="-420" y="30" z="-90" />
            <point x="-250" y="24" z="-80" />
            <point x="0" y="18" z="-70" />
            <point x="260" y="14" z="-65" />
            <point x="420" y="10" z="-60" />
            <point x="200" y="-180" z="-180" />
            <point x="-300" y="-170" z="-170" />
        </translate>
        <rotate time="2.5" x="1" y="1" z="0" />
        <scale x="0.55" y="0.55" z="0.55" />
    </transform>
    <models>
        <model file="asteroid.3d" color="#8A8178" cull="false" />
    </models>
</group>

```

Listagem 13: Declaração do cometa com trajetória elíptica e alinhamento com a tangente.

4.3.2. Cinturões de asteróides

Para enriquecer a cena e aumentar a sensação de escala e dinamismo do sistema solar animaram-se os dois cinturões de asteróides.

Apesar de modelados como malhas estáticas, foram animados através de uma rotação contínua em torno do Sol. Esta rotação é aplicada em torno do eixo $(0, 0, 1)$, simulando o movimento orbital coletivo dos corpos que compõem estes cinturões.

Foram definidos períodos de rotação distintos para reforçar a diferença de escala entre os dois:

O cinturão de asteróides possui um período de rotação de 170 segundos, apresentando um movimento relativamente rápido e facilmente perceptível; O cinturão de Kuiper possui um período de 11000 segundos, resultando num movimento extremamente lento, coerente com a sua maior distância ao Sol.

Embora cada cinturão seja representado como uma única geometria contínua (em vez de múltiplos objetos individuais), esta abordagem permite obter um efeito visual convincente com baixo custo computacional. A rotação global da malha cria a ilusão de um conjunto denso de corpos em órbita, contribuindo para a complexidade visual da cena sem impactar significativamente o desempenho.

4.4. Geração Procedimental de Órbitas

De forma a melhorar a qualidade visual das órbitas planetárias, foi desenvolvido um *script* auxiliar em *Python* que gera automaticamente os pontos de controlo das curvas de Catmull-Rom utilizadas no ficheiro XML.

Inicialmente, as órbitas eram definidas com um número reduzido de pontos, o que resultava em trajetórias pouco suaves e com desvios visíveis relativamente a uma circunferência. Para resolver este problema, optou-se por gerar um conjunto denso de pontos distribuídos ao longo da órbita.

```
function make_points(major_axis, eccentricity, samples):
    minor_axis = major_axis * sqrt(1 - e^2)

    for i in range(samples):
        ang = 2π * i / samples
        x = a * (cos(ang) - e)
        z = b * sin(ang)
        pts.append((x, 0, z))

    pts.extend(pts[:3]) # fechar curva
    return pts
```

Listagem 14: Geração de pontos ao longo de uma órbita elíptica.

As órbitas são modeladas como elipses com o Sol localizado num dos focos, utilizando os parâmetros do semi-eixo maior (a) e da excentricidade (e). O semi-eixo menor (b) é calculado através da relação $b = a\sqrt{1 - e^2}$.

Os valores de excentricidade utilizados para cada planeta foram aproximados com base em dados reais do Sistema Solar, permitindo representar órbitas elípticas mais realistas em vez de trajetórias perfeitamente circulares, onde a excentricidade define o grau de “achatamento” da órbita [6].

Para cada planeta, o script gera um número fixo de amostras igualmente espaçadas no ângulo, produzindo uma discretização uniforme da órbita. Este aumento da densidade

de pontos permite que a interpolação de Catmull-Rom produza trajetórias significativamente mais suaves e visualmente mais próximas de circunferências.

Adicionalmente, são repetidos os três primeiros pontos no final da lista, de forma a garantir a continuidade da curva ao longo de todo o percurso, evitando descontinuidades quando a animação completa um ciclo.

O script percorre automaticamente o ficheiro XML e substitui os blocos de translação correspondentes a cada planeta, permitindo atualizar todas as órbitas de forma consistente e eficiente.

Esta abordagem permitiu separar a geração geométrica das órbitas da lógica da *engine*, simplificando o código e facilitando a experimentação com diferentes níveis de detalhe.

Na versão final do sistema, foram utilizados 120 pontos de amostragem por órbita, garantindo uma aproximação suficientemente suave das trajetórias elípticas sem comprometer o desempenho da aplicação.

4.5. Resultados Obtidos

A cena do sistema solar animado foi renderizada com sucesso, exibindo:

- O Sol ao centro, a rodar em torno do seu eixo;
- Os oito planetas e Plutão a orbitar em curvas Catmull-Rom, com velocidades proporcionais aos períodos reais;
- A Lua da Terra a orbitar o planeta em curva independente, herdando o movimento orbital da Terra pela hierarquia de grupos;
- Os objetos estáticos todos a beneficiar do ganho de desempenho proporcionado pelos VBOs;
- Os cometas a seguir a sua trajetória elíptica e alongada com alinhamento com a tangente.

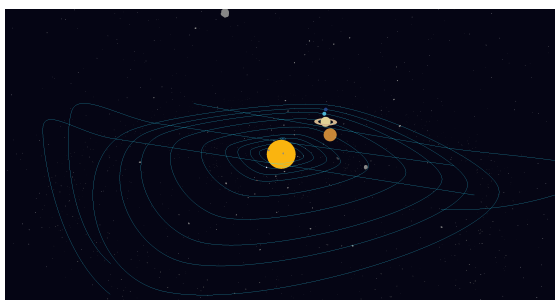


Figura 3: Resultado 1 — curvas de Catmull-Rom com 7 pontos.

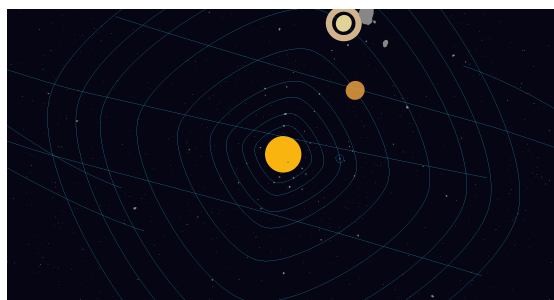


Figura 4: Resultado 2 — curvas de Catmull-Rom com 7 pontos, vista de cima.

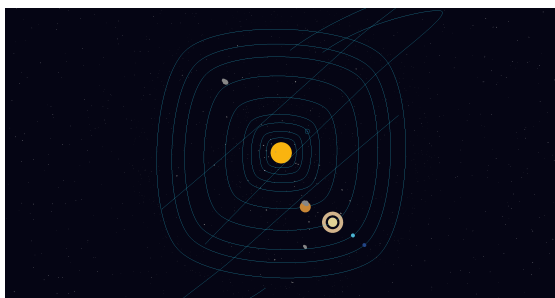


Figura 5: Resultado 3 — curvas de Catmull-Rom com 7 pontos, vista de cima.

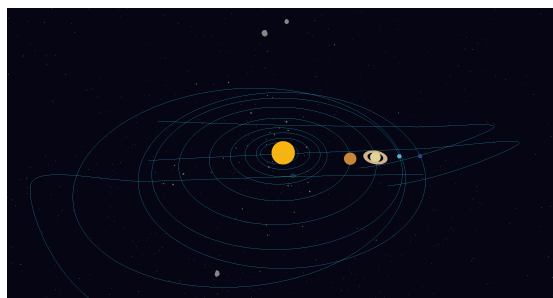


Figura 6: Resultado 4 — curvas de Catmull-Rom com 120 pontos, vista de cima.

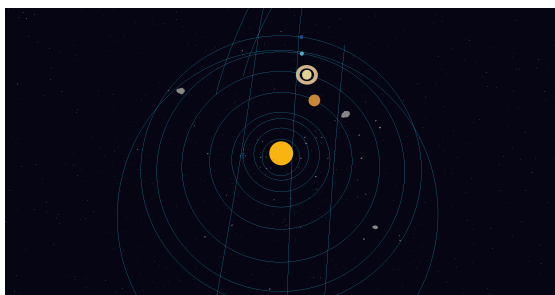


Figura 7: Resultado 5 — curvas de Catmull-Rom com 120 pontos, vista de cima.

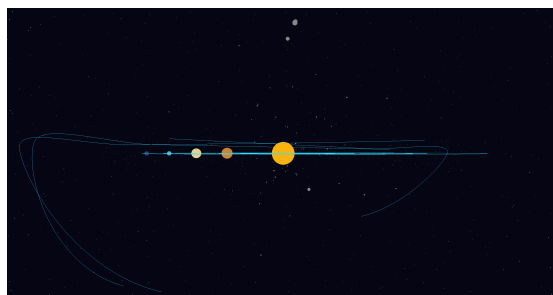


Figura 8: Resultado 6 — curvas de Catmull-Rom com 120 pontos, vista lateral.

Para avaliar o impacto da utilização de VBOs na performance, foi efetuada uma comparação entre execução com e sem utilização destes *buffers* à versão do modelo estático do sistema solar.

Sem VBOs, a aplicação apresenta um tempo médio de frame de aproximadamente 23.95 ms, com oscilações mais visíveis durante a renderização da cena.

Frame	Timestamp (ms)	Frametime (ms)
0	1240	34
1	1261	21
2	1287	26
3	1309	22
4	1331	22
5	1351	20

Tabela 3: Amostra de medições de frametime sem VBOs.

Com a utilização de VBOs, observa-se uma melhoria significativa no desempenho, com o tempo médio de frame reduzido para cerca de 8.28 ms. A variação entre frames torna-se também mais estável, refletindo uma maior eficiência na gestão dos dados gráficos.

Estes resultados demonstram que a utilização de VBOs reduz significativamente o custo de envio de dados para a GPU, permitindo uma renderização mais eficiente e consistente, especialmente em cenas com múltiplos objetos em movimento.

A melhoria observada confirma a importância da utilização de *buffers* no contexto de OpenGL para aplicações com animação contínua e múltiplos modelos simultâneos.

Frame	Timestamp (ms)	Frametime (ms)
0	1249	60
1	1251	2
2	1258	7
3	1263	5
4	1266	3
5	1268	2

Tabela 4: Amostra de medições de frametime com VBOs.

5. Conclusão

A terceira fase do trabalho prático foi concluída com sucesso. Os dois grandes objetivos da fase — implementação de VBOs e implementação de transformações dependentes do tempo — foram atingidos, e o cenário demonstrativo do sistema solar animado foi produzido.

Do lado do *generator*, a adição do suporte a *patches* de Bézier bicúbicas abre caminho para a geração de geometria muito mais complexa e expressiva do que as primitivas simples das fases anteriores.

Do lado da *engine*, a migração para VBOs eliminou o principal *bottleneck* de desempenho. A implementação das curvas de Catmull-Rom com suporte a alinhamento com a tangente permite criar animações de trajetória expressivas e naturais, e a distinção entre transformações estáticas e animadas na estrutura de dados manteve o código organizado e a compatibilidade retroativa com as cenas XML das fases anteriores.

Para a fase seguinte, que introduz iluminação, materiais e texturas, a base da *engine* já permite a utilização eficiente de VBOs para armazenamento de vértices. No entanto, nesta fase, os VBOs contêm apenas informação de posição, não estando ainda implementado o suporte para normais nem coordenadas de textura.

Assim, o trabalho futuro passará por estender as estruturas de dados e o processo de carregamento para incluir estes atributos adicionais, bem como atualizar o *pipeline* de renderização para os utilizar corretamente.

Paralelamente, será implementado o modelo de iluminação de Phong, recorrendo às fontes de luz definidas no XML, permitindo enriquecer significativamente o realismo visual da cena.

Bibliografia

- [1] Khronos Group, «OpenGL - The Industry Standard for High Performance Graphics». [Em linha]. Disponível em: <https://www.opengl.org/>
- [2] G. Farin, *Curves and Surfaces for CAGD: A Practical Guide*, 5.ª ed. San Francisco: Morgan Kaufmann, 2002.
- [3] E. Catmull e R. Rom, «A Class of Local Interpolating Splines», *Computer Aided Geometric Design*. Academic Press, New York, pp. 317–326, 1974.
- [4] Wikipedia, «Orbital Period». [Em linha]. Disponível em: https://en.wikipedia.org/wiki/Orbital_period
- [5] Wikipedia, «Rotation Period». [Em linha]. Disponível em: [https://en.wikipedia.org/wiki/Rotation_period_\(astronomy\)](https://en.wikipedia.org/wiki/Rotation_period_(astronomy))
- [6] BBC- Sky At Night Magazine, «Orbital Orbital eccentricity». [Em linha]. Disponível em: <https://www.skyatnightmagazine.com/space-science/orbital-eccentricity>

Lista de Siglas e Acrónimos

API Application Programming Interface

CCW Counter-Clockwise

CPU Central Processing Unit

FOV Field of View

GPU Graphics Processing Unit

VAO Vertex Array Object

VBO Vertex Buffer Objects

XML Extensible Markup Language

Anexos

Anexo 1: Logo da Universidade do Minho

